Übungsblatt 6

Asynchrones JavaScript, Browser-Speicher & Web-APIs

5430UE Web and Data Engineering

20. Mai 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs

Lernziele

- Grundlegendes Verständnis von asynchronem JavaScript.
- Einblick in die Verarbeitung externer Daten über die Fetch API.
- Grundlegendes Verständnis von Web Workers und clientseitiger Speicherung.

Währungsrechner - Fetch API und asynchrones JavaScript

Währungsrechner - Fetch API und asynchrones JavaScript

Implementieren Sie einen Währungsrechner, der Beträge von Euro (EUR) in US-Dollar (USD) oder Schweizer Franken (CHF) umrechnet. Das Projekt *waehrungsrechner* ist in Stud.IP hinterlegt. Die Dateien *index.html* und *style.css* sind bereits vollständig funktionsfähig und müssen nicht verändert werden.

Vervollständigen Sie die *script.js* an den markierten Stellen.

- Bauen Sie die URL dynamisch mit den Parametern `amount`, `from=EUR` und `to` (Zielwährung) zusammen.
- Wandeln Sie die API-Antwort in JSON um und extrahieren Sie den umgerechneten Wert aus dem Feld `data.rates`.
- Genauere Informationen zur Form der URL und der Struktur des Responses finden Sie unter Frankfurter API v1
- Geben Sie das Ergebnis formatiert (2 Nachkommastellen) im Element `#result` aus.
- Registrieren Sie die notwendigen EventListener, damit sich das Ergebnis bei jeder Eingabe oder Auswahl automatisch aktualisiert.

Produkte laden - Fetch API und asynchrones JavaScript

Produkte laden - Fetch API und asynchrones JavaScript

Implementieren Sie die Funktion `loadProduct(id)`, um Produktdetails von einer externen API abzurufen und anzuzeigen.

- *Abruf*: Rufen Sie mittels fetch die URL `https://api.example.com/products/{id}` auf (GET-Request).
- *Validierung*: Prüfen Sie `response.ok`. Werfen Sie bei Fehlern (Netzwerk oder HTTP-Status >= 400) eine Exception.
- *Rendering*: Wandeln Sie das Ergebnis in JSON um und geben Sie die Felder `name`, `price` und `description` im Element `<div id="product-detail">` aus.
- *Fehlerbehandlung*: Fangen Sie alle Fehler mit `try...catch` ab und zeigen Sie eine entsprechende Fehlermeldung im selben `<div>` an.
- Verwenden Sie konsequent die `async/await`-Syntax.

Hinweis: Die gegebene API-Adresse stellt lediglich eine Dummy-Adresse dar und liefert kein Ergebnis.



Web Workers und Local Storage

Web Workers und Local Storage

1. **Web Workers:**
    1. Erklären sie kurz, was man unter einem *Web Worker* versteht.
    2. Ein Online-Shop berechnet Produktempfehlungen per komplexem Scoring-Algorithmus. Nach der Implementierung friert die UI kurz ein. Erklären Sie: Warum friert die UI ein? Welche Technologie löst dieses Problem?
2. **Szenario:**

    Ein Nutzer füllt ein mehrstufiges Registrierungsformular aus. Die bereits eingegebenen Daten sollen bei einem versehentlichen Neuladen der Seite erhalten bleiben, nach dem Schließen des Tabs jedoch verworfen werden. Zusätzlich soll die vom Nutzer gewählte Spracheinstellung auch nach einem Browser-Neustart erhalten bleiben.

    Welche Möglichkeiten bietet Web Storage zur clientseitigen Persistenz? Vergleichen Sie zwei davon hinsichtlich:
    - Lebensdauer (bis wann gehen Daten verloren?)
    - Speicherkapazität (ungefähr)
    - Typischer Einsatz

Hinweis zu den Übungssitzungen am 20.05.2026: Projektauftakt

Hinweis zu den Übungssitzungen am 20.05.2026: Projektauftakt

In den Sitzungen am 20.05.2026 werden neben der Besprechung der Übungsaufgaben von Blatt 06 auch die Projektaufgabe vorgestellt sowie organisatorische Hinweise zum Projekt besprochen. Die Teilnahme an einer dieser Sitzungen ist verpflichtende Voraussetzung für die Teilnahme am Projekt.

Hinweis: Die Übungsgruppe von 10–12 Uhr war bislang durchgehend gut besucht, sodass zu erwarten ist, dass auch nächste Woche nur noch wenige Sitzplätze frei sind. Teilnehmende, die bisher noch nicht an den Übungen teilgenommen haben, werden daher gebeten, nach Möglichkeit bevorzugt die Sitzung von 8–10 Uhr zu besuchen.

Materialien

Materialien zum Übungsblatt

- Aufgabe *Währungsrechner - Fetch API und asynchrones JavaScript*: waehrungsrechner.zip